

Объектно- Ориентированное Программирование

(МИСиС, зима-весна 2026)

Полевой Дмитрий Валерьевич

к.т.н., доцент КиК

teams: 8url2s2

e-mail: dvpsun@gmail.com

tg: @dvpsun

ваши вопросы

Тестирование шаблонного кода

- тест шаблона
 - руками написать какие типы тестируем
- шаблонный тест
 - `TEST_CASE_TEMPLATE`
- шаблонный тест шаблона

Руками подставить тип шаблона

```
TEST_CASE (" [arrayt<float>] - ctor" ) {  
    CHECK ( ArrayT<float> () .size () == 0 );  
}
```

```
TEST_CASE (" [arrayt<int>] - ctor" ) {  
    CHECK ( ArrayT<int> () .size () == 0 );  
}
```

Шаблонный тест

```
TEST_CASE_TEMPLATE("[arrayt] - ctor",  
T, int, float, std::string) {  
    ArrayT<T> a;  
    CHECK(0 == a.size());  
}
```

Как унифицировать список

```
#define TYPES int, float, std::string  
  
TEST_CASE_TEMPLATE("ctor copy", T, TYPES) {
```

Как унифицировать тест

- вынести зависящие от типа части в шаблонные функции
- написать специализации (шаблона) для тестируемых типов

Шаблонный тест

(разделение определения и применения)

- TEST_CASE_TEMPLATE_DEFINE
- TEST_CASE_TEMPLATE_INVOKE
- TEST_CASE_TEMPLATE_APPLY

Если нужны сочетания типов (привести самым к ОДНОМУ параметру)

```
template <typename first, typename second>  
struct TP {  
    typedef first A;  
    typedef second B;  
};
```

Если нужны сочетания типов (привести самым к ОДНОМУ параметру)

```
#define pairs TP<int, char>, TP<char, int>
```

```
TEST_CASE_TEMPLATE("multiple types", T,  
pairs) {
```

```
    typedef typename T::A T1;
```

```
    typedef typename T::B T2;
```

```
    // use T1 and T2 types
```

```
}
```

Доработка основного кода

- по мере тестирования с разными типами могут появляться ошибки - исправлять
- по мере понимания ограничений на типы параметров – добавлять проверки ограничений в код (проверки времени компиляции – `static assert`)

ваши вопросы